

Project 03 예약 SaaS — Phase 1-3 학습정리

도메인 엔티티 설계: 리치 도메인 + 상속 2단 구조 (멀티테넌시 빼대)

이 문서는 "내가 한 것" + "남들이 자주 겪는 문제 + 우리 케이스"를 함께 수록한다.

0. Phase 1-3은 무엇인가

1-2에서 클린 아키텍처 4계층 골격(빈 프로젝트)을 세웠다. 1-3에서는 그 가장 안쪽(Domain)에 첫 코드 — 엔티티를 만든다. 인증·멀티테넌시에 필요한 최소 엔티티(`Tenant` , `User`)와, 그것들의 공통 부모(`BaseEntity` , `TenantEntity`)를 설계했다.

1-3에서 한 일 - 공통 부모 2단 상속 구조 설계 (`BaseEntity` → `TenantEntity`) - `Tenant` , `User` 엔티티를 리치 도메인 방식으로 작성 (규칙을 엔티티에 캡슐화) - 관계는 단방향(`Tenant.Users`)만 — 실용적 판단 - 빌드 검증

아직 안 한 것: xmin 낙관적 잠금은 엔티티 속성이 아니라 EF Core 설정이라 1-D에서, DB 테이블 생성(마이그레이션)도 1-D에서.

1. 상속 2단 구조 — 왜 BaseEntity와 TenantEntity를 나눴나

```
BaseEntity      ← Id, CreatedAt (모든 엔티티 공통)
  ▲ 상속
TenantEntity    ← + TenantId (테넌트에 "속하는" 엔티티만)
  ▲ 상속
User, Customer... ← TenantEntity 상속 (TenantId 자동 보유)

Tenant          ← BaseEntity만 상속 (자기는 테넌트에 안 속함 → TenantId 없음)
```

- `BaseEntity` : `Id` , `CreatedAt` . 모든 엔티티의 최상위 부모. `abstract` .
- `TenantEntity` : `BaseEntity` 상속 + `TenantId` . "테넌트에 소속되는" 엔티티들의 부모. `abstract` .
- `Tenant` (실제 회사): `BaseEntity` 만 상속. 자기 자신은 테넌트에 소속될 수 없으므로 `TenantId` 가 없음.
- `User` (사용자): `TenantEntity` 상속 → `Id` + `CreatedAt` + `TenantId` 자동 보유.

왜 2단으로 나눴나 (면접 포인트): 모든 엔티티를 하나의 BaseEntity에 `TenantId` 까지 넣으면, `Tenant` 가 자기 `TenantId` 를 갖는 모순이 생긴다. "테넌트에 속하는 것"과 "안 속하는 것"을 타입으로 구분하면, 1-F에서 " `TenantEntity` 를 상속한 것에만 테넌트 필터 적용"을 깔끔하게 걸 수 있다. 이것이 "실용적 상속"의 핵심.

2. 리치 도메인 vs 빈약한 도메인

	리치 도메인 (rich)	빈약한 도메인 (anemic)
형태	생성자 검증 + <code>private set</code> + 변경 메서드	속성만 <code>{ get; set; }</code>
잘못된 상태	객체 존재 = 규칙 지킴. 원천 차단	서비스가 깜빡하면 뚫림
규칙 위치	엔티티에 응집	서비스에 흩어짐
예	03 <code>Tenant</code> , <code>User</code>	IPlus <code>Item_PatientInfo</code> (DTO)

우리 선택: 핵심 엔티티(`Tenant` , `User` , 이후 `Reservation`)는 리치, 단순 조회/설정용은 가볍게. 전부 리치는 오버엔지니어링이므로 하지 않는다.

"anemic(빈혈의)" = 로직 없이 데이터만 있는 강통. 클린 아키텍처를 표방하면서 엔티티가 강통이면 "구조만 나누고 알맹이는 그대로"가 된다. 리치 도메인이 진짜 차별점.

3. 만든 코드와 핵심 문법

BaseEntity / TenantEntity

```
namespace Reservation.Domain.Entities;

public abstract class BaseEntity
{
    public Guid Id { get; set; }
    public DateTime CreatedAt { get; set; }
}

public abstract class TenantEntity : BaseEntity
{
    public Guid TenantId { get; set; }
}
```

Tenant (리치)

```
public class Tenant : BaseEntity
{
    public string Name { get; private set; } = null!;
    public ICollection<User> Users { get; private set; } = new List<User>();

    private Tenant() { } // EF Core 전용

    public Tenant(string name)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("테넌트 이름은 필수입니다.", nameof(name));
        Id = Guid.NewGuid();
        CreatedAt = DateTime.UtcNow;
        Name = name.Trim();
    }

    public void Rename(string newName)
    {
        if (string.IsNullOrEmpty(newName))
            throw new ArgumentException("테넌트 이름은 비울 수 없습니다.", nameof(newName));
        Name = newName.Trim();
    }
}
```

User (리치, TenantEntity 상속)

```
public class User : TenantEntity
{
    public string Email { get; private set; } = null!;
    public string PasswordHash { get; private set; } = null!;

    private User() { }

    public User(Guid tenantId, string email, string passwordHash)
    {
        if (tenantId == Guid.Empty) throw new ArgumentException("TenantId는 필수입니다.", nameof(tenantId));
        if (string.IsNullOrEmpty(email)) throw new ArgumentException("이메일은 필수입니다.", nameof(email));
        if (string.IsNullOrEmpty(passwordHash)) throw new ArgumentException("비밀번호 해시는 필수입니다.", nameof(passwordHash));

        Id = Guid.NewGuid();
        CreatedAt = DateTime.UtcNow;
        TenantId = tenantId;
        Email = email.Trim().ToLowerInvariant();
        PasswordHash = passwordHash;
    }
}
```

핵심 문법 사전

문법	의미	왜 쓰나
<code>abstract class</code>	직접 객체 생성 불가, 상속 전용	BaseEntity/TenantEntity는 공통 묶음이지 실제 대상이 아님
<code>: BaseEntity</code>	상속(부모 지정)	콜론 뒤가 부모. 부모 속성을 물려받음
<code>{ get; private set; }</code>	읽기는 공개, 쓰기는 클래스 내부만	바깥에서 직접 변경 차단 → 메서드로만 변경(캡슐화)
<code>private 생성자()</code>	EF Core 전용 빈 생성자	EF가 DB→객체 복원 시 빈 생성자 선호. private이라 우리 코드에선 못 씀
<code>= null!</code>	null 용서 연산자	"지금은 null이나 실제로 채워짐"을 컴파일러에 보장(경고만 끄)
<code>nameof(x)</code>	변수명을 문자열로	예외 메시지에 파라미터명. 이름 바뀌면 자동 반영
Guid 기본키	전역 고유 ID	테넌트별 ID 충돌 없음 + ID 추측 방지(보안)
<code>public</code> (엔티티)	다른 프로젝트에서 접근 가능	엔티티는 계층(프로젝트) 넘어 공유되므로 internal 불가

4. null 용서 연산자 ! — 제대로 이해하기

- ! 는 컴파일러의 null 경고만 끄는 표시("null-forgiving"). "내가 값 채울 책임을 진다"는 약속이지 강제가 아니다.
- 약속을 어기면(실제로 값을 안 채우면) 컴파일은 조용하지만 실행 중 `NullReferenceException` 이 터진다.
- 정당한 사용: `Email = null!` 은 리치 생성자와 EF Core가 반드시 값을 채우므로 안전.
- 위험한 사용: 근거 없이 "경고 귀찮으니" 남발 → 진짜 null 유입 경로를 가려 버그를 심는다.
- 판단 기준: "여기가 왜 null 아님을 보장하지?"에 답할 수 있으면 ! 사용 OK.

5. ★ 흔한 함정 (남들이 자주 겪는 것 + 우리 케이스)

#	함정	증상	우리 케이스 / 예방
1	<code>ToLower()</code> 문화권 의존 (터키어 i)	터키어 로케일에서 <code>I</code> → <code>ı</code> (점 없는 i). 가입/로그인 이메일 불일치로 "가입했는데 로그인 안 됨"	정규화·비교용 문자열은 <code>ToLowerInvariant()</code> . 우리 이메일 정규화에 적용
2	<code>DateTime.Now</code> 사용	서버 지역 시간이 저장됨(우리 서버는 westus2=미국!). 시간 꼬임	저장은 <code>DateTime.UtcNow</code> 로 통일, 표시할 때만 지역 변환
3	엔티티를 <code>internal</code> 로 방치	다른 프로젝트(Application/Infra)에서 "형식 없음" 컴파일 에러	계층 넘어 공유되는 엔티티는 <code>public</code>
4	<code>= null!</code> 의 ! 누락	"Null 리터럴을 null 비허용 형식으로 변환 불가" 경고	<code>= null</code> → <code>= null!</code> (우리도 이 경고 겪음)
5	리치 엔티티에 빈 생성자 없음	EF Core가 객체 복원 실패(런타임)	<code>private 엔티티() { }</code> 빈 생성자 추가
6	아직 없는 타입 참조	<code>Tenant</code> 가 <code>User</code> 를 참조하는데 <code>User</code> 미생성 → "형식 없음"	정상. 상호 참조라 둘 다 만들 때까지 빌드 안 됨
7	비밀번호 평문 저장	유출 시 즉시 계정 탈취	속성명을 <code>PasswordHash</code> 로. 평문 저장 안 함(해싱은 1-E)
8	관계를 무조건 양방향으로	불필요한 EF 설정·복잡도 증가	필요할 때만. 우리는 <code>Tenant.Users</code> 단방향만

6. 일관성 원칙 — 이 프로젝트를 관통하는 사고

"환경/상황에 따라 달라지는 것을 없앤다"가 반복된다. - 1-A: DB·테이블 이름 **소문자 통일** (PostgreSQL 대소문자 함정) - 1-C: 이메일 `ToLowerInvariant()` 정규화 (문화권 함정) - 1-C: 시간 `UtcNow` 통일 (서버 지역 함정) - 1-C: 기본키 `Guid` (테넌트별 ID 충돌 방지)

면접: "환경에 독립적이고 일관된 데이터"를 위해 정규화·UTC·Guid를 일관 적용했다고 설명 가능.

7. IPlus(실무)와의 대비

	IPlus	03
모델	<code>Item *</code> = 속성만 있는 DTO(빈약)	<code>Tenant / User</code> = 규칙 품은 리치 도메인
검증	Service에 흩어짐	엔티티 생성자/메서드에 응집
잘못된 상태	서비스가 놓치면 유입	객체 생성 자체가 차단

IPlus의 계층형 + DTO 경험 위에, "도메인 모델에 규칙을 캡슐화"를 더한 것이 03. 구조(1-2)뿐 아니라 알맹이(1-3)도 모던화.

8. 면접에서 말할 수 있는 포인트

1. **상속 2단 설계** — 테넌트 소속 여부를 타입으로 구분(BaseEntity vs TenantEntity). 필터 일괄 적용의 토대.
2. **리치 도메인** — 불변식을 생성자에서 강제, `private set` 으로 캡슐화. 잘못된 상태를 구조로 차단.
3. **EF Core와의 절충** — private 빈 생성자, `null!` 등 ORM 제약을 이해하고 리치 도메인과 공존.
4. **일관성 설계** — Invariant 정규화 / UTC / Guid로 환경 독립성 확보.
5. **관계 판단** — 양방향을 기본으로 걸지 않고 필요에 따라 단방향 선택.
6. **보안 기본기** — 비밀번호는 `PasswordHash` 로만, 평문 저장 배제.

9. 다음 — Phase 1-4 (EF Core 연결 + 첫 마이그레이션)

Domain 엔티티를 실제 DB 테이블로 연결한다. - Infrastructure에 **DbContext** 작성, EF Core 10 + Npgsql 10.x 패키지(패밀리 통일). - 엔티티 → 테이블 매핑, **xmin 낙관적 잠금** (`UseXminAsConcurrencyToken`) 설정. - **첫 마이그레이션** 생성 + 적용. 컨테이너 시작 시 `db.Database.Migrate()` 자동 적용 패턴. - `reservationdb` 에 실제 테이블이 생기는 것 확인(1-A에서 만든 전용 유저로 연결, 교체한 새 암호 사용).

Phase 1-3 학습정리 끝. 작성 기준: .NET 10 / C# / Reservation.Domain.